

KFA Public Library wants to digitise its catalogue and membership system. Over the next three hours you will build it up feature by feature, section by section — each section adds a new layer on top of the previous one, exactly like a real software project grows. Treat this as one continuous Java project, not four unrelated exercises.

Create a single project named KFALibrary. Keep every class you write; you will keep extending it through Sections A–D.

General Instructions

- Work individually. You may use your IntelliJ IDE, the JDK documentation, and your own class notes — **no AI tools**.
- Comment your code briefly wherever the logic isn't obvious; uncommented "magic" logic loses clarity marks.
- Each section builds on the one before it — do not delete earlier classes, only extend them.
- Compile and test each class before moving to the next section.
- Submit one zipped folder named RollNo_KFALibrary.zip containing all .java files and a screenshot folder of each program's console output.
- Upload before the 12 noon of the assigned Tutorial day.

Marks Distribution

Section	Chapter
A	Java & OOP Foundations (Unit 1 & 2)
B	Inheritance & Interfaces (Unit 3)
C	Exception Handling & Packages (Unit 4)
D	Strings (Unit 5)

Section A

A1. The Book blueprint

- Create a class Book with private fields: title, author, isbn (String), price (double), available (boolean).
- Write a parameterised constructor that sets all fields except available, which should always start as true.
- Add getters and setters for every field, applying sensible validation in the setter for price (reject negative values).
- Add a static field totalBooks that increments in the constructor, and a static method getTotalBooks().
- Override toString() to print the book neatly, e.g. "[B001] Clean Code by Robert Martin — Rs 850.00 (Available)".
- In a Main class, create at least 4 Book objects and print them using a loop, followed by the total book count.

A2. Concept check (short written answers, 2–3 sentences each)

- Using your Book class as the example, explain what encapsulation buys you here that a class with public fields would not.
- Explain the difference between a class and an object, again anchored in your Book example ("class" vs one specific book on the shelf).
- Why did we make totalBooks static instead of an instance field? What would go wrong if it weren't static?

Section B

KFA's library doesn't just lend books — it also lends Magazines and DVDs. Rather than duplicating fields, refactor Book into a family of related item types.

B1. Building the hierarchy

- Create an abstract class `LibraryItem` holding the common fields from `Book` (title, isbn, price, available) and an abstract method `getLendingPeriodDays()` that returns how many days an item may be borrowed for.
- Make `Book` extend `LibraryItem` (books lend for 14 days). Create two new subclasses: `Magazine` (7 days) and `DVD` (5 days), each adding one field of its own (e.g. `Magazine` has `issueNumber`, `DVD` has `durationMinutes`).
- Override `toString()` in each subclass so the output clearly differs, and override `getLendingPeriodDays()` appropriately in each.

B2. Interfaces and renewability

- Create an interface `Renewable` with a method `renew(int extraDays)` that extends the lending period.
- Implement `Renewable` in `Book` and `DVD` only (Magazines cannot be renewed in this library's policy) — explain in a one-line comment why `Magazine` deliberately does not implement it.

B3. Polymorphism

- In `Main`, build a `LibraryItem[]` array (or `ArrayList`) holding a mix of `Book`, `Magazine`, and `DVD` objects.
- Loop through it calling `toString()` and `getLendingPeriodDays()` on each — without any `instanceof` or casting — and explain in one sentence why this works.

Section C

A real library system must cope with members trying to borrow a book that's out on loan, or returning something late. It also needs to be organised, not one giant pile of files.

C1. Custom exceptions

- Create a checked exception class `BookNotAvailableException` with a constructor that accepts and stores a message.
- Create a second checked exception class `ItemOverdueException` carrying the number of days overdue.
- Add a method `borrowItem(LibraryItem item)` in a new `LibrarySystem` class that throws `BookNotAvailableException` if the item's available flag is false, and otherwise marks it unavailable.
- Add a method `returnItem(LibraryItem item, int daysLate)` that throws `ItemOverdueException` when `daysLate > 0`, and otherwise marks the item available again.

C2. Catching gracefully

- In `Main`, call `borrowItem()` and `returnItem()` inside try-catch-finally blocks that catch your custom exceptions and print a friendly message (not a raw stack trace).
- Use the finally block to print a confirmation line such as "Transaction processed for: <title>" regardless of whether an exception occurred.
- Demonstrate at least one successful transaction and one that triggers each custom exception.

C3. Packages

- Reorganise your classes into three packages: `kfa.model` (`LibraryItem`, `Book`, `Magazine`, `DVD`, `Renewable`), `kfa.exception` (your two exception classes), and `kfa.service` (`LibrarySystem`).
- Add the correct package statements and import statements so everything compiles, and keep `Main` in the default package.

Section D

The final piece: turning raw text into something the library system can use — generating IDs, validating input, and producing readable reports.

D1. Member ID generator

- Write a static method `generateMemberId(String fullName)` that: takes the first 3 letters of the first name (uppercase), takes the first 2 letters of the last name (uppercase), and appends a random 3-digit number — e.g. "Aarav Shrestha" → "AARSH482".
- Handle the edge case of a single-word name (no last name) sensibly, and explain your handling in a comment.
- Use only String methods taught in class (`substring`, `toUpperCase`, `split`, `length`, `trim`) — do not use regex.

D2. ISBN sanity check

- Write a static method `isValidIsbn(String isbn)` that returns true only if the string is exactly 13 characters long, contains only digits, and does not start with '0'.
- Use String methods such as `length()`, `charAt()`, and a loop — no regex.
- Test it against at least 4 sample ISBNs, including at least one that should fail each individual rule.

D3. Catalogue report with `StringBuilder`

- Write a method `buildCatalogueReport(LibraryItem[] items)` that uses `StringBuilder` to construct one neatly formatted multi-line report string listing every item's title and availability.
- Add a simple search: given a keyword, use String methods (`contains`, `toLowerCase`) to filter and list only titles that match, without creating a new array — build the filtered text straight into the `StringBuilder`.
- Explain in 2–3 sentences why `StringBuilder` is preferred over plain String concatenation (`+=`) inside this loop, referencing string immutability.

Submission Checklist

- All .java files compile with zero errors.
- Package structure matches Section C3.
- Console screenshots included for Sections A, B, C, and D runs.
- Section A2 written answers saved as `answers.txt` or `answers.docx` inside the same zip.
- Zip named `RollNo_KFALibrary.zip` and uploaded before time runs out.

Work steadily through A → D in order — each section is designed to take roughly the suggested time, so pacing yourself matters more than perfecting Section A. Good luck!